

Wael balhoudi
classe :MP ASSCC1

I. Introduction

Dans le cadre du module d'administration des systèmes et réseaux, ce projet consiste à concevoir et déployer une infrastructure complète simulant un environnement d'entreprise.

L'objectif est de centraliser plusieurs services critiques :

- l'authentification des utilisateurs
- le stockage et partage de fichiers
- la gestion des logs
- l'envoi de notifications par email
- l'automatisation des tâches

Cette architecture repose sur plusieurs machines virtuelles interconnectées afin de reproduire une infrastructure réaliste, sécurisée et évolutive.

II. Objectifs

Les principaux objectifs du projet sont :

- Mettre en place un système d'authentification centralisé avec LDAP
 - Déployer une plateforme de stockage avec OwnCloud
 - Configurer un serveur de messagerie avec Postfix
 - Centraliser les journaux système avec Graylog
 - Automatiser les alertes et rapports avec N8N
 - Garantir la communication sécurisée entre les services
-

III. Architecture du système

L'infrastructure est composée de trois machines virtuelles :

1. VM1 — Client

- Poste utilisateur
 - Accès aux services via navigateur web
 - Test des fonctionnalités (connexion, upload, etc.)
 - addres
-

2. VM2 — Serveur Applicatif

Cette machine héberge plusieurs services essentiels :

- OwnCloud : stockage et partage de fichiers
 - Postfix : serveur SMTP (relais d'emails)
 - Graylog : centralisation et analyse des logs
-

3. VM3 — Serveur LDAP

- Héberge OpenLDAP
 - Gère l'authentification centralisée
 - Fournit les identités utilisateurs aux autres services
-

4. Service externe

- Brevo : utilisé comme relais SMTP pour l'envoi d'emails vers l'extérieur
-

IV. Fonctionnement global

Le système fonctionne comme suit :

1. L'utilisateur accède à OwnCloud depuis la machine cliente
 2. OwnCloud vérifie les identifiants via le serveur LDAP
 3. Une fois authentifié, l'utilisateur peut manipuler ses fichiers
 4. Toutes les actions sont enregistrées dans les logs
 5. Les logs sont envoyés vers Graylog pour analyse
 6. En cas d'événement important, une notification peut être générée
 7. Postfix envoie les emails via Brevo
 8. (Extension) N8N automatise la génération de rapports et alertes
-

V. Mise en œuvre

1. Configuration LDAP (VM3)

Etap1 : Préparation système & Installation d'OpenLDAP

```
sudo apt update && sudo apt upgrade -y
```

```
sudo hostnamectl set-hostname ldap.asscc1.tn
```

```
echo "192.168.1.101 ldap.asscc1.tn ldap" | sudo tee -a /etc/hosts
```

```
sudo apt install -y slapd ldap-utils openssl apparmor-utils
```

```
sudo dpkg-reconfigure slapd
```

Etap2 : Certificats TLS (Structure blindée)

1 create a new folder called ssl-aldap and go to it

```
mkdir ssl-ldap  
cd mkdir ssl-ldap
```

2 generate self signed ssl certificate:

```
openssl genrsa -out ldap.asscc1.tn.key 4096  
chmod 600 ldap.asscc1.tn.key
```

```
openssl rsa -in ldap.asscc1.tn.key -out ldap.asscc1.tn.key
```

```
openssl req -new -days 3650 -key ldap.asscc1.tn.key -out ldap.asscc1.tn.csr
```

```
sudo openssl x509 -in ldap.asscc1.tn.csr -out ldap.asscc1.tn.crt -req -  
signkey ldap.asscc1.tn.key -days 3650
```

3: configure SSL on openLDAP Server

3.1 copy certificates and key to /etc/ldap/sasl2 directory

```
sudo cp ldap.asscc1.tn.key /etc/ldap/sasl2/  
sudo cp ldap.asscc1.tn.crt /etc/ldap/sasl2/  
sudo cp /etc/ssl/certs/ca-certificates.crt /etc/ldap/sasl2/
```

3.2 change the ownership of the certificates to openldap user

```
sudo chown -R openldap:openldap /etc/ldap/sasl2/
```

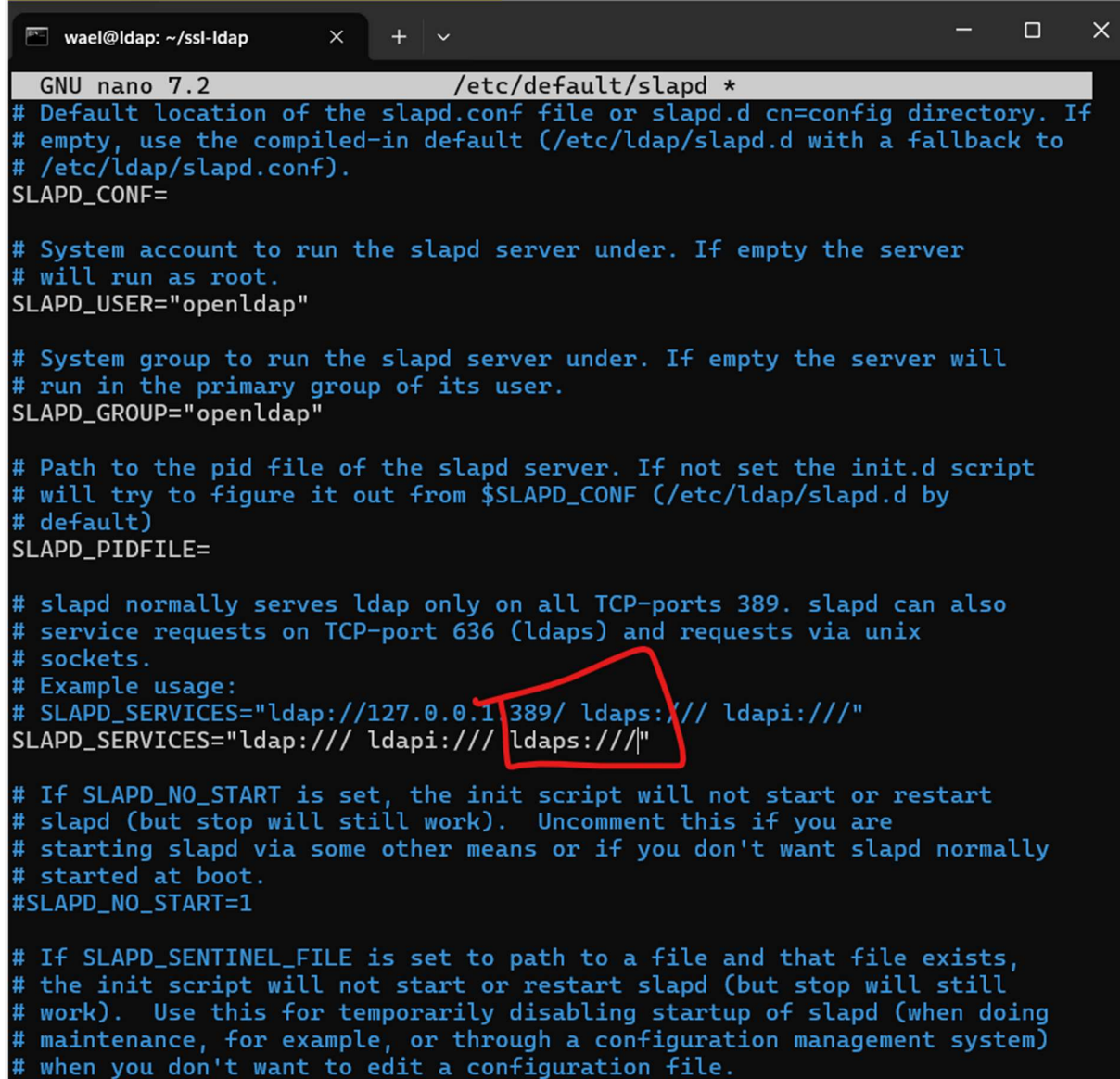
3.3 create LDAP configuration file called SSL-LDAP.ldif

```
nano SSL-LDAP.ldif
```

```
dn: cn=config  
changetype: modify  
add: olcTLSCACertificateFile  
olcTLSCACertificateFile: /etc/ldap/sasl2/ca-certificates.crt  
-  
replace: olcTLSCertificateFile  
olcTLSCertificateFile: /etc/ldap/sasl2/ldap.asscc1.tn.crt
```

```
-  
replace: olcTLSCertificateKeyFile  
olcTLSCertificateKeyFile: /etc/ldap/sasl2/ldap.asscc1.tn.key  
  
sudo ldapmodify -Y EXTERNAL -H ldapi:/// -f SSL-LDAP.ldif
```

```
3.5 edit /etc/default/slapd file  
sudo nano /etc/default/slapd file
```



```
GNU nano 7.2 /etc/default/slapd *  
# Default location of the slapd.conf file or slapd.d cn=config directory. If  
# empty, use the compiled-in default (/etc/ldap/slapd.d with a fallback to  
# /etc/ldap/slapd.conf).  
SLAPD_CONF=  
  
# System account to run the slapd server under. If empty the server  
# will run as root.  
SLAPD_USER="openldap"  
  
# System group to run the slapd server under. If empty the server will  
# run in the primary group of its user.  
SLAPD_GROUP="openldap"  
  
# Path to the pid file of the slapd server. If not set the init.d script  
# will try to figure it out from $SLAPD_CONF (/etc/ldap/slapd.d by  
# default)  
SLAPD_PIDFILE=  
  
# slapd normally serves ldap only on all TCP-ports 389. slapd can also  
# service requests on TCP-port 636 (ldaps) and requests via unix  
# sockets.  
# Example usage:  
# SLAPD_SERVICES="ldap://127.0.0.1:389/ ldaps:/// ldapi:///"  
SLAPD_SERVICES="ldap:/// ldapi:/// ldaps:///"  
  
# If SLAPD_NO_START is set, the init script will not start or restart  
# slapd (but stop will still work). Uncomment this if you are  
# starting slapd via some other means or if you don't want slapd normally  
# started at boot.  
#SLAPD_NO_START=1  
  
# If SLAPD_SENTINEL_FILE is set to path to a file and that file exists,  
# the init script will not start or restart slapd (but stop will still  
# work). Use this for temporarily disabling startup of slapd (when doing  
# maintenance, for example, or through a configuration management system)  
# when you don't want to edit a configuration file.
```

```
3.6 edit/etc/ldap/ldap.conf  
sudo nano /etc/ldap/ldap.conf
```

```
wael@ldap: ~/ssl-ldap
GNU nano 7.2 /etc/ldap/ldap.conf *
#
# LDAP Defaults
#
# See ldap.conf(5) for details
# This file should be world readable but not world writable.

#BASE    dc=example,dc=com
#URI      ldap://ldap.example.com ldap://ldap-provider.example.com:666

#SIZELIMIT    12
#TIMELIMIT    15
#DEREF        never

# TLS certificates (needed for GnuTLS)
#TLS_CACERT    /etc/ssl/certs/ca-certificates.crt
TLS_CACERT    /etc/ldap/sasl2/ca-certificates.crt
TLS_REQCERT    allow
```

"""

```
    TLS_CACERT    /etc/ldap/sasl2/ca-certificates.crt
    TLS_REQCERT    allow
```

"""

```
/// command I don't know what do
sudo ldapmodify -Y EXTERNAL -H ldapi:/// <<EOF
dn: cn=config
changetype: modify
add: olcDisallows
olcDisallows: bind_anon
EOF
```

3.7 now , restart LDAP service to apply the changes

```
sudo systemctl restart slapd
```

4:verify

4.1 verify be ldapsearch command

```
ldapsearch -H ldaps://192.168.1.101 \
-D "cn=admin,dc=asscc1,dc=tn" \
-W \
-b "dc=asscc1,dc=tn" \
"(objectClass=*)"
```

```

wael@ldap:~/ssl-ldap$ ldapsearch -H ldaps://192.168.1.101 \
-D "cn=admin,dc=asscc1,dc=tn" \
-W \
-b "dc=asscc1,dc=tn" \
"(objectClass=*)"
Enter LDAP Password:
# extended LDIF
#
# LDAPv3
# base <dc=asscc1,dc=tn> with scope subtree
# filter: (objectClass=*)
# requesting: ALL
#
# asscc1.tn
dn: dc=asscc1,dc=tn
objectClass: top
objectClass: dcObject
objectClass: organization
o: asscc1
dc: asscc1

# search result
search: 2
result: 0 Success

# numResponses: 2
# numEntries: 1
wael@ldap:~/ssl-ldap$ |

```

4.2 verify by port connectivity

```
netstat -antup |grep -I 636
```

```

wael@ldap:~/ssl-ldap$ netstat -antup |grep -I 636
(No info could be read for "-p": geteuid()=1000 but you should be root.)
tcp        0      0 0.0.0.0:636          0.0.0.0:*           LISTEN
-
tcp6       0      0 :::636              :::*                 LISTEN
-
wael@ldap:~/ssl-ldap$

```

5. configure firewall

```
wael@ldap: ~/ssl-ldap
wael@ldap:~/ssl-ldap$ sudo ufw enable
Command may disrupt existing ssh connections. Proceed with operation (y|n)?
y
Firewall is active and enabled on system startup
wael@ldap:~/ssl-ldap$ sudo ufw allow 636
Rule added
Rule added (v6)
wael@ldap:~/ssl-ldap$ sudo ufw reload
Firewall reloaded
wael@ldap:~/ssl-ldap$ sudo ufw status
Status: active

To Action From
--
636 ALLOW Anywhere
636 (v6) ALLOW Anywhere (v6)

wael@ldap:~/ssl-ldap$ |
```

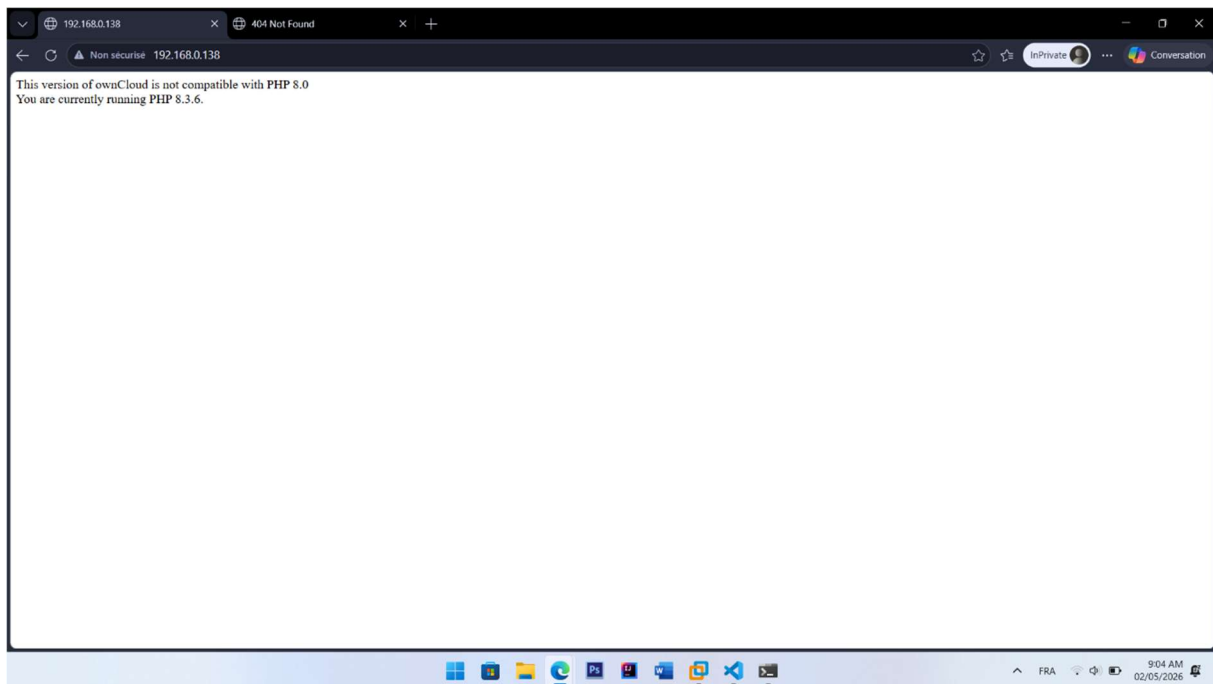
2. Déploiement de OwnCloud (VM2)

Dans ce projet, deux méthodes principales d'installation d'OwnCloud sont possibles :

- Installation directe sur le système (via Apache/Nginx et PHP)
- Utilisation de conteneurs Docker

J'ai choisi la **méthode Docker** pour plusieurs raisons techniques et pratiques

- Isolation de l'application pour éviter les conflits avec le système hôte
- Gestion simplifiée des dépendances (PHP, base de données, modules déjà inclus)
- Installation rapide sans configuration manuelle complexe
- Portabilité du service entre différentes machines ou environnements
- Déploiement et suppression faciles avec Docker Compose



1. Install Docker

sudo apt update

sudo apt install docker.io docker-compose -y

sudo systemctl enable docker

sudo systemctl start docker

2 create docker compose file

nano docker-compose.yml

```
wael@wael: ~/owncloud-docl x + v
GNU nano 7.2 docker-compose.yml
services:
  owncloud:
    image: owncloud/server:10.13
    container_name: owncloud
    restart: always
    ports:
      - "8080:8080"
    environment:
      - OWNCLOUD_ADMIN_USERNAME=admin
      - OWNCLOUD_ADMIN_PASSWORD=strongpassword
      - OWNCLOUD_TRUSTED_DOMAINS=192.168.1.100:8080
    volumes:
      - owncloud_data:/mnt/data

volumes:
  owncloud_data:
```


services:

owncloud:

image: owncloud/server:10.13

container_name: owncloud

restart: always

ports:

- "8080:8080"

environment:

- OWNCLOUD_ADMIN_USERNAME=admin

- OWNCLOUD_ADMIN_PASSWORD=strongpassword

- OWNCLOUD_TRUSTED_DOMAINS=192.168.1.100:8080

volumes:

- owncloud_data:/mnt/data

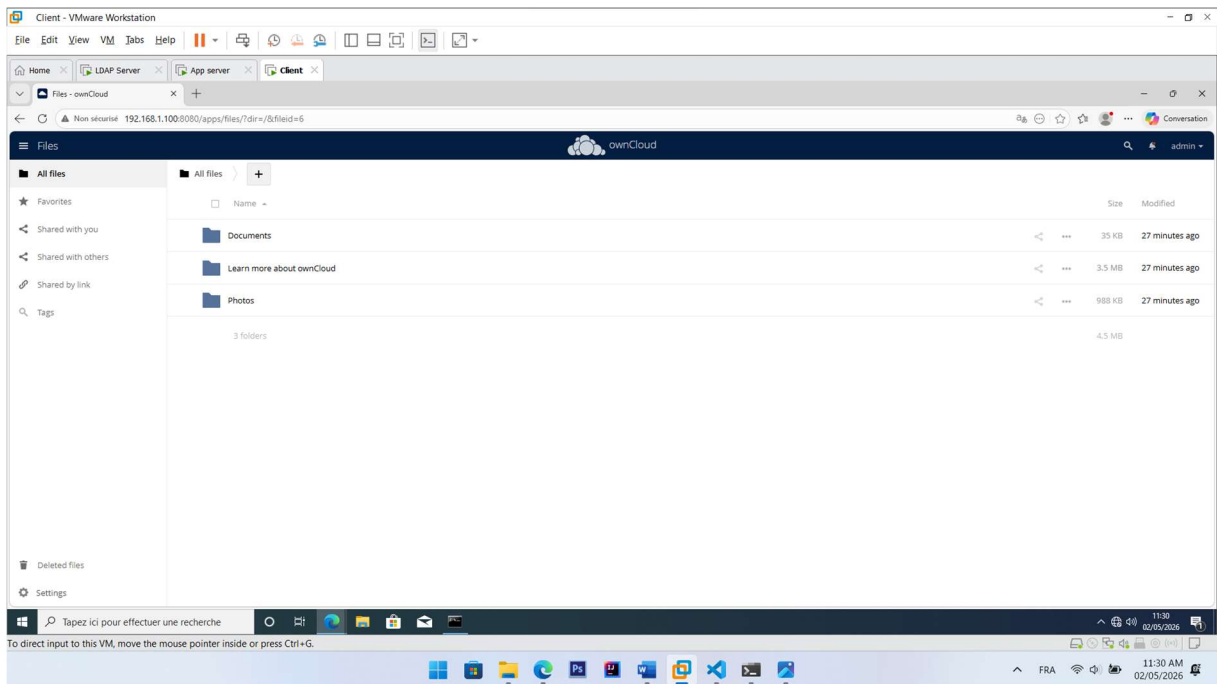
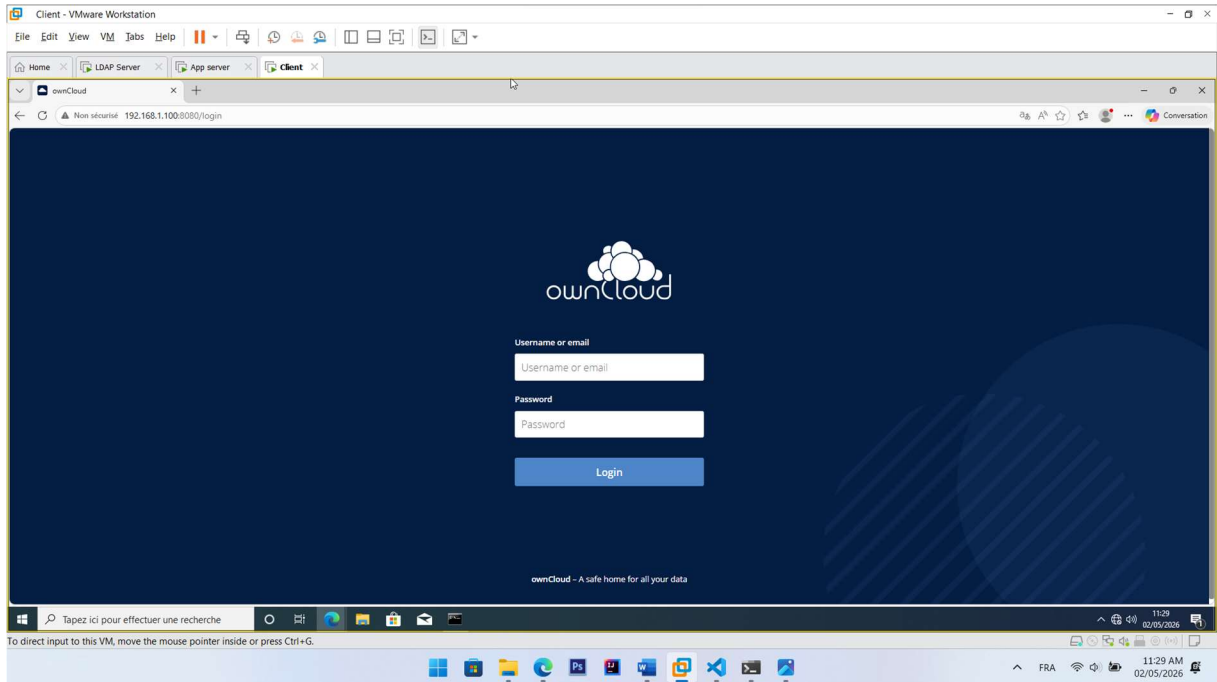
volumes:

owncloud_data:

3. Run the container

`docker compose up -d`

4.test on the browser



3. Connect OwnCloud → LDAP

create test users on VM3 — Serveur LDAP

3.1 create an LDIF file: `nano add-users.ldif`

Create People OU

dn: ou=people,dc=asscc1,dc=tn

objectClass: organizationalUnit

ou: people

Test User 1

dn: uid=testuser1,ou=people,dc=asscc1,dc=tn

objectClass: inetOrgPerson

objectClass: posixAccount

objectClass: shadowAccount

uid: testuser1

cn: Test User One

sn: User One

givenName: Test

mail: testuser1@asscc1.tn

uidNumber: 10001

gidNumber: 10001

homeDirectory: /home/testuser1

loginShell: /bin/bash

userPassword: {SSHA}test123

Test User 2

dn: uid=testuser2,ou=people,dc=asscc1,dc=tn

objectClass: inetOrgPerson

objectClass: posixAccount

objectClass: shadowAccount

uid: testuser2

cn: Test User Two

sn: User Two

givenName: Test

mail: testuser2@asscc1.tn

uidNumber: 10002

gidNumber: 10002

homeDirectory: /home/testuser2

loginShell: /bin/bash

userPassword: {SSHA}test456

3.2 add user to LDAP

```
ldapadd -x -H ldap://192.168.1.101:389 \
```

```
-D "cn=admin,dc=asscc1,dc=tn" -W \
```

```
-f add-users.ldif
```

3.3 verify users exit

```
ldapsearch -x -H ldap://192.168.1.101:389 \
```

```
-D "cn=admin,dc=asscc1,dc=tn" -W \
```

```
-b "dc=asscc1,dc=tn" \
```

"(objectClass=inetOrgPerson)" uid cn mail

```
wael@ldap: ~  
wael@ldap:~$ ldapsearch -x -H ldap://192.168.1.101:389 \  
-D "cn=admin,dc=asscc1,dc=tn" -W \  
-b "dc=asscc1,dc=tn" \  
"(objectClass=inetOrgPerson)" uid cn mail  
Enter LDAP Password:  
# extended LDIF  
#  
# LDAPv3  
# base <dc=asscc1,dc=tn> with scope subtree  
# filter: (objectClass=inetOrgPerson)  
# requesting: uid cn mail  
#  
# testuser1, people, asscc1.tn  
dn: uid=testuser1,ou=people,dc=asscc1,dc=tn  
uid: testuser1  
cn: Test User One  
mail: testuser1@asscc1.tn  
  
# testuser2, people, asscc1.tn  
dn: uid=testuser2,ou=people,dc=asscc1,dc=tn  
uid: testuser2  
cn: Test User Two  
mail: testuser2@asscc1.tn  
  
# search result  
search: 2  
result: 0 Success  
  
# numResponses: 3  
# numEntries: 2  
wael@ldap:~$ |
```

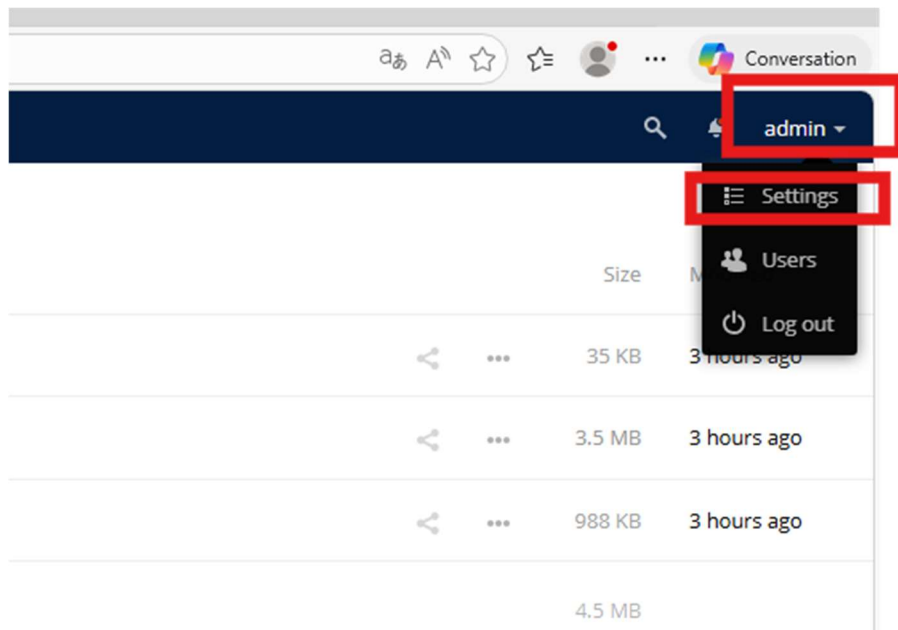
3.4 configure OwnCloud (VM2)

Step 1: Access OwnCloud Settings

Open browser: <http://192.168.1.100:8080>

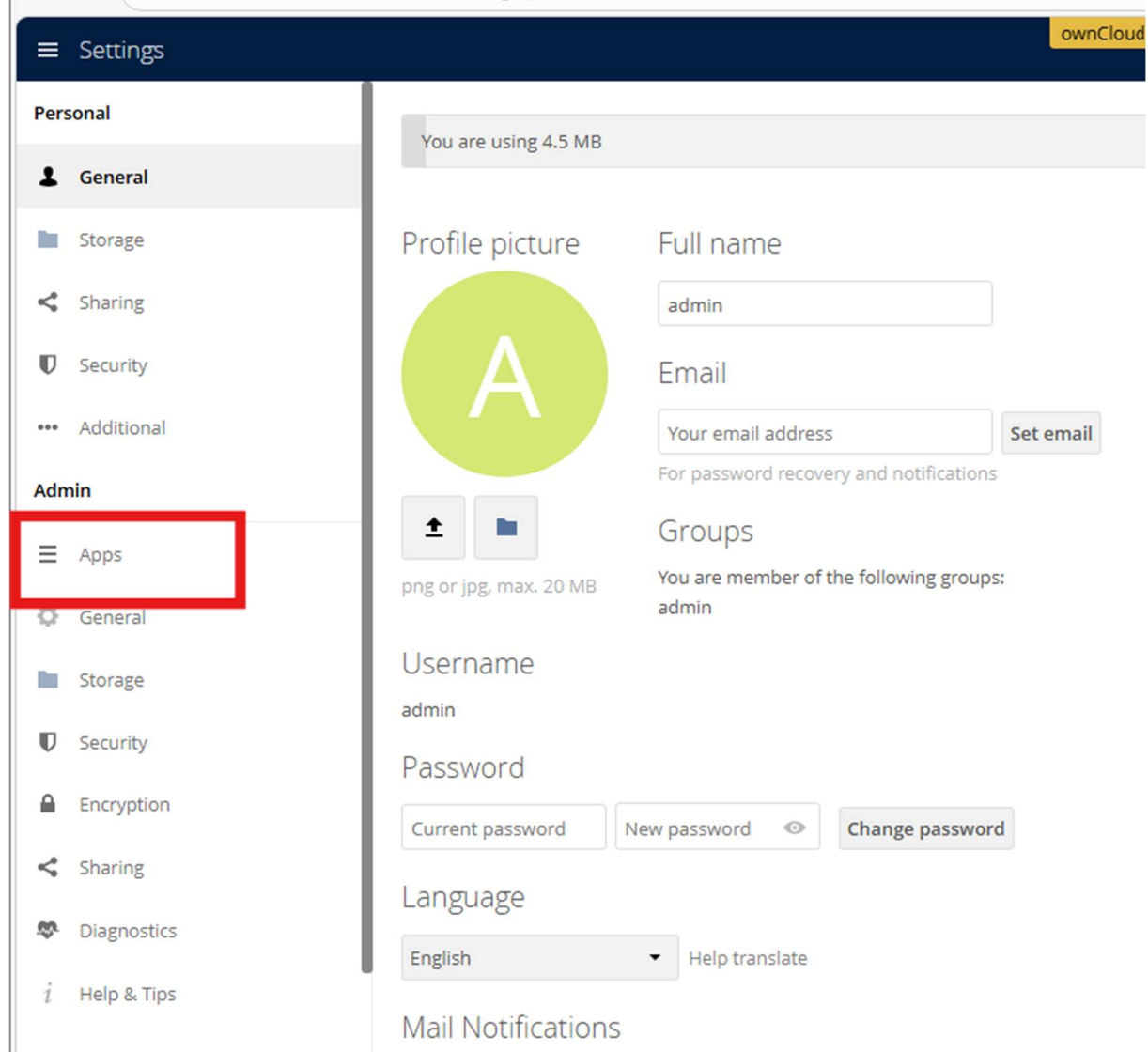
Login as admin (your OwnCloud admin, not LDAP)

Click your username (top-right) → Settings



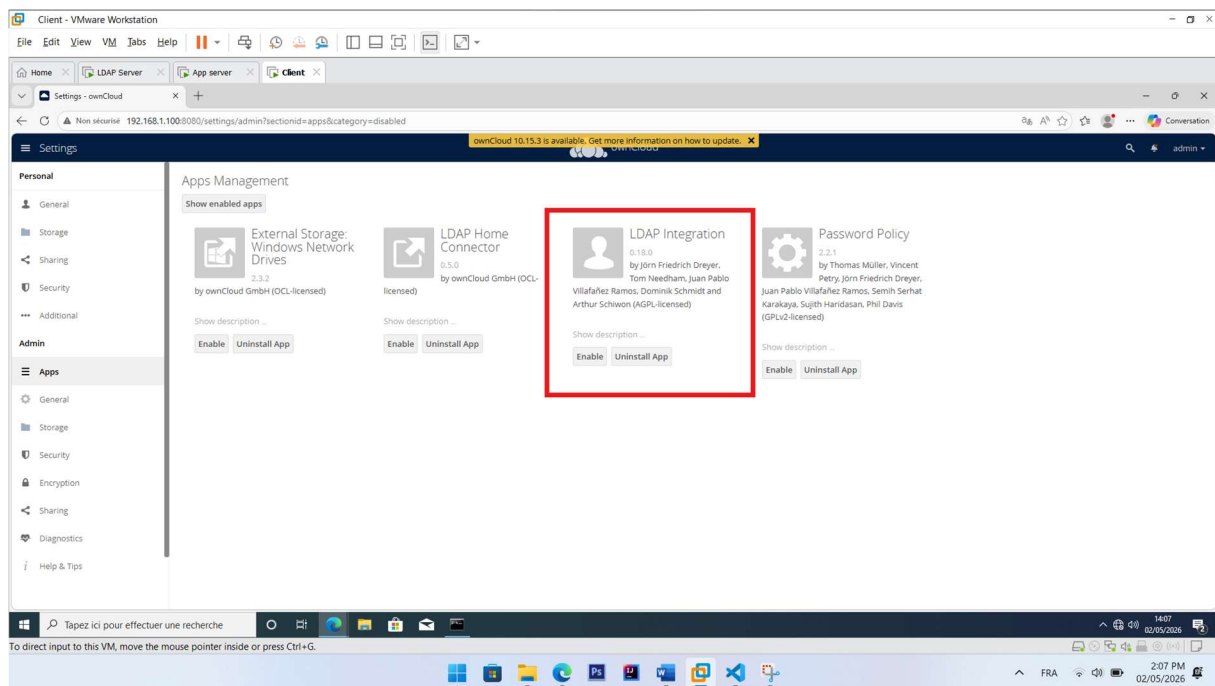
Step 2: Enable LDAP App

In left menu, click Apps



Search for "LDAP"

Find "LDAP Integration"



Click Enable (if not already enabled)

Go to: Settings → User Authentication

Field in Screenshot	Value to Enter
Host	192.168.1.101
Port	636
Use StartTLS support	Leave Unchecked
User DN	cn=admin,dc=asscc1,dc=tn
Password	(Enter your LDAP Admin Password here)
One Base DN per line	dc=asscc1,dc=tn

Go to Users tab

Configure:

Only these object classes: inetOrgPerson

Edit LDAP Query: (&(objectClass=inetOrgPerson)(uid=*))

Click "Verify settings and count users" → Should show 2 users

LDAP

Server

Users

Login Attributes

Groups

Advanced

Expert

Configuration OK
192.168.1.101:389

Help

ownCloud access is limited to users meeting these criteria:

Only these object classes:

inetOrgPerson

2

i

The most common object classes for users are organizationalPerson, person, user, and inetOrgPerson. If you are not sure which object class to select, please consult your directory admin.

Only from these groups:

Search groups

Select groups

0

[Edit LDAP Query](#)

Edit LDAP Query

{{(objectclass=inetOrgPerson)}}

i

The filter specifies which LDAP users shall have access to the ownCloud instance.

Verify settings and count users

2 users found

Configuration OK
192.168.1.101:389

Back

Continue

Test user login

User found and settings verified.

LDAP

Server Users **Login Attributes** Groups Advanced Expert Configuration OK 192.168.1.101:389 Help

When logging in, ownCloud will find the user based on the following attributes:

- ☒ LDAP / AD Username:
Allows login against the LDAP / AD username, which is either uid or samaccountname and will be detected.
- ☐ LDAP / AD Email Address:
Allows login against an email attribute. Mail and mailPrimaryAddress will be allowed. WARNING: Disabling login with email might require enabling strict login checking to be effective, please refer to ownCloud documentation for more details!

Other Attributes:

[Edit LDAP Query](#)

Edit LDAP Query

`(&(objectClass=inetOrgPerson)(uid=%uid))`

Defines the filter to apply, when login is attempted. %uid replaces the username in the login action. Example: "uid=%uid"

Test Loginname

Attempts to receive a DN for the given loginname and the current login filter.

Configuration OK 192.168.1.101:389

Step 3: Test Real Login

Logout from OwnCloud:

Click your username (top-right corner) → Logout

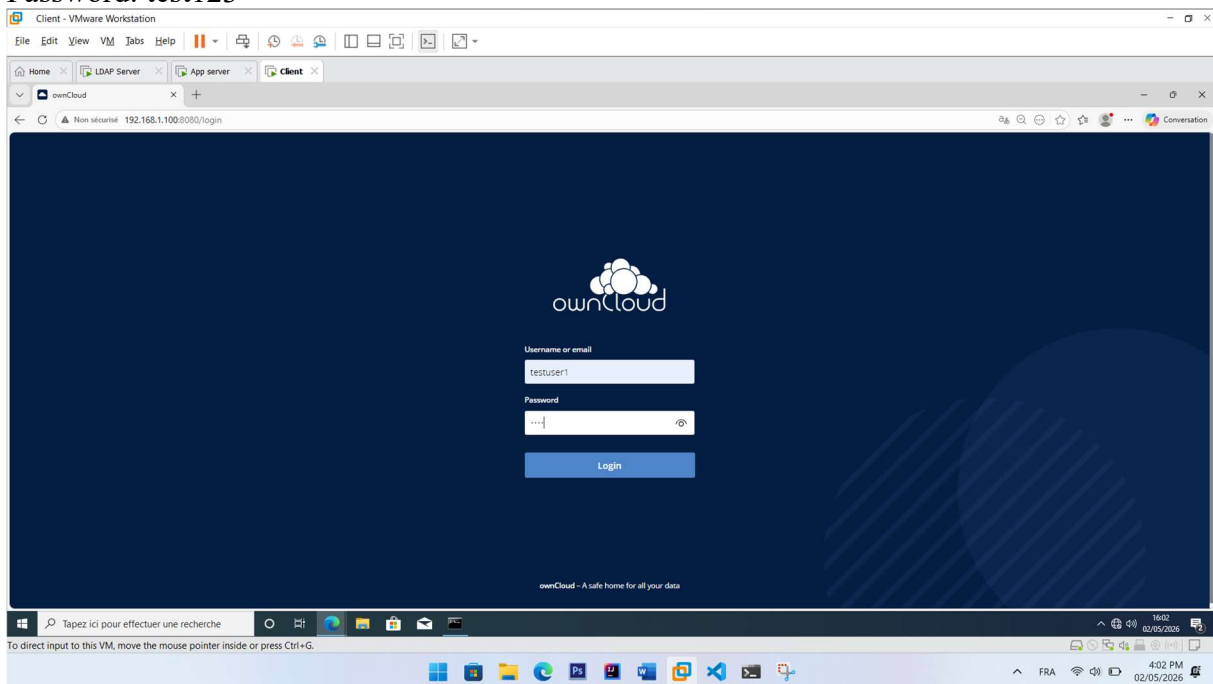
Open a new incognito/private window in your browser

Go to: <http://192.168.1.100:8080>

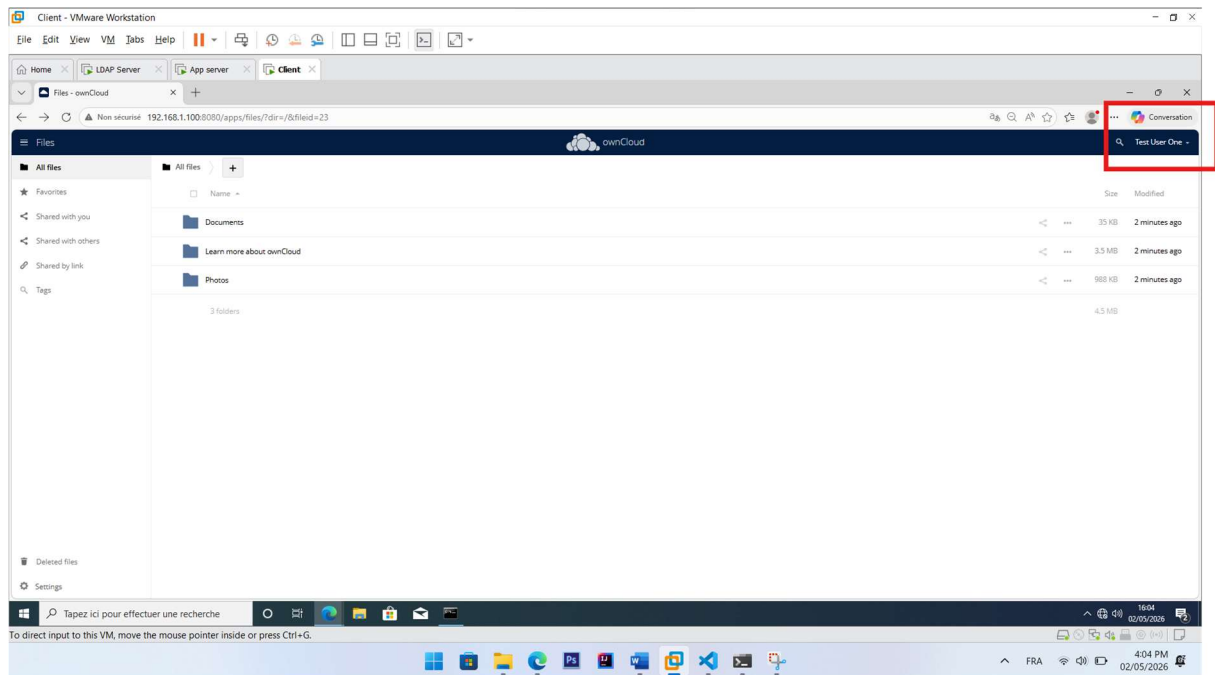
Login with LDAP user:

Username: testuser1

Password: test123

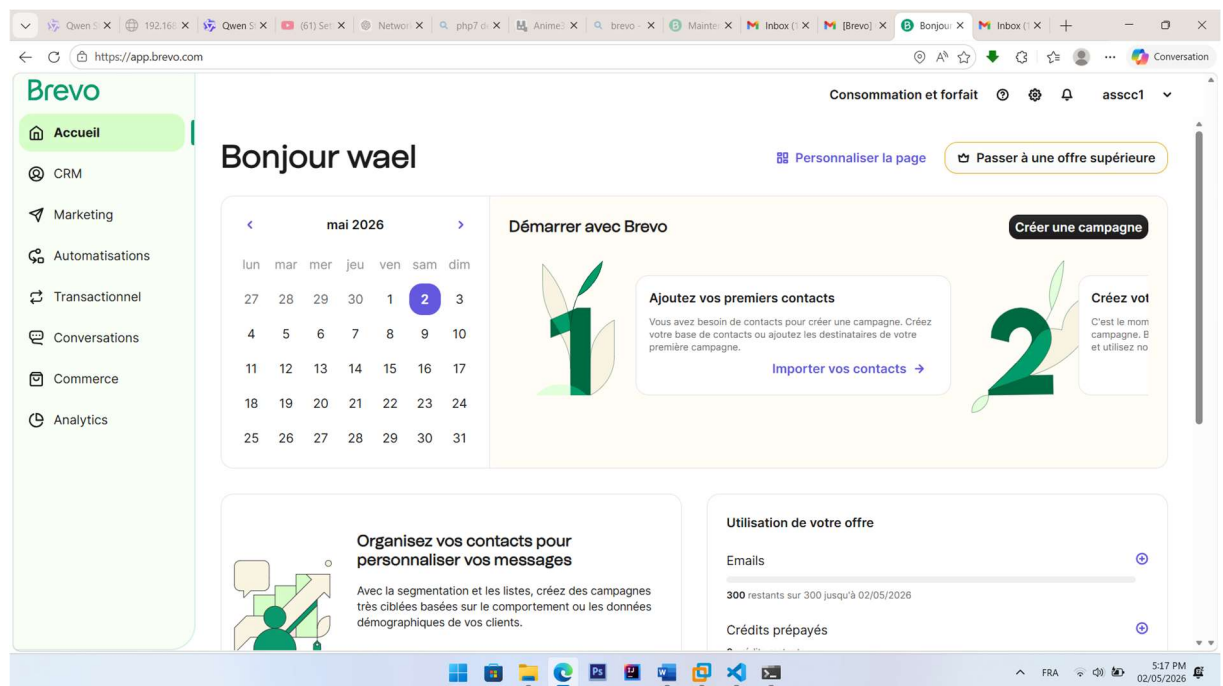


☒ Success: You'll be logged in as testuser1

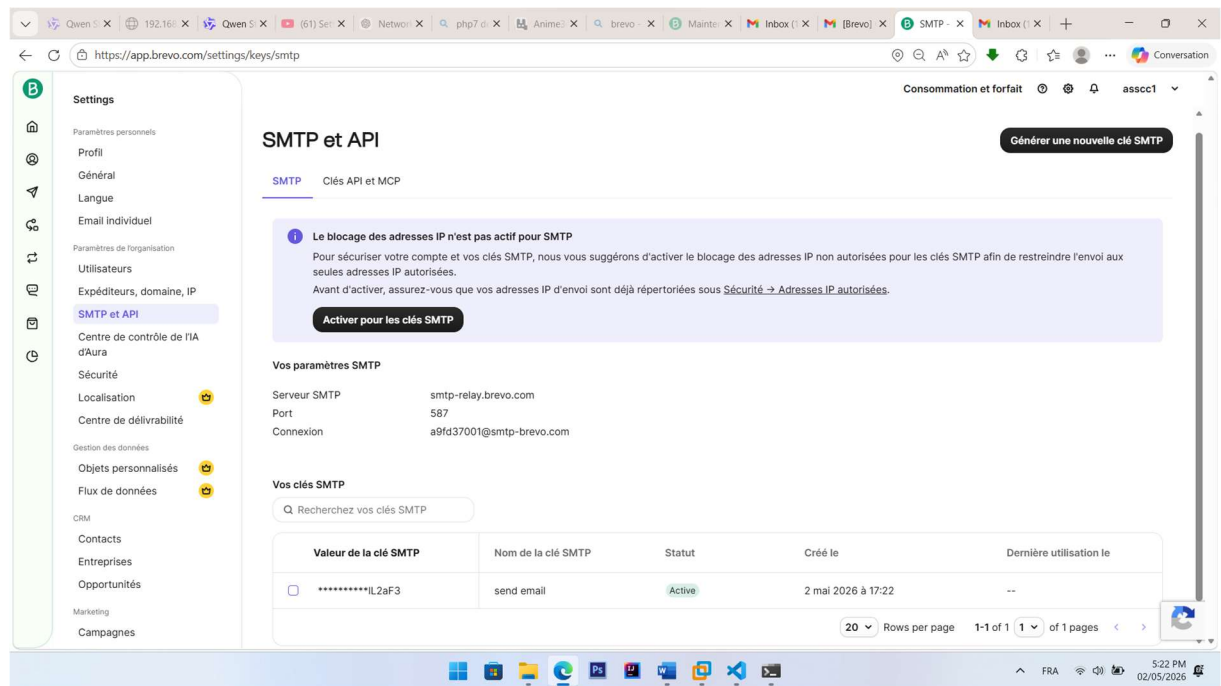


4. Configuration de Postfix (VM2)

Step1 :create Brevo account



Step 2: Get SMTP Credentials



Step 3: Configure Postfix on VM2

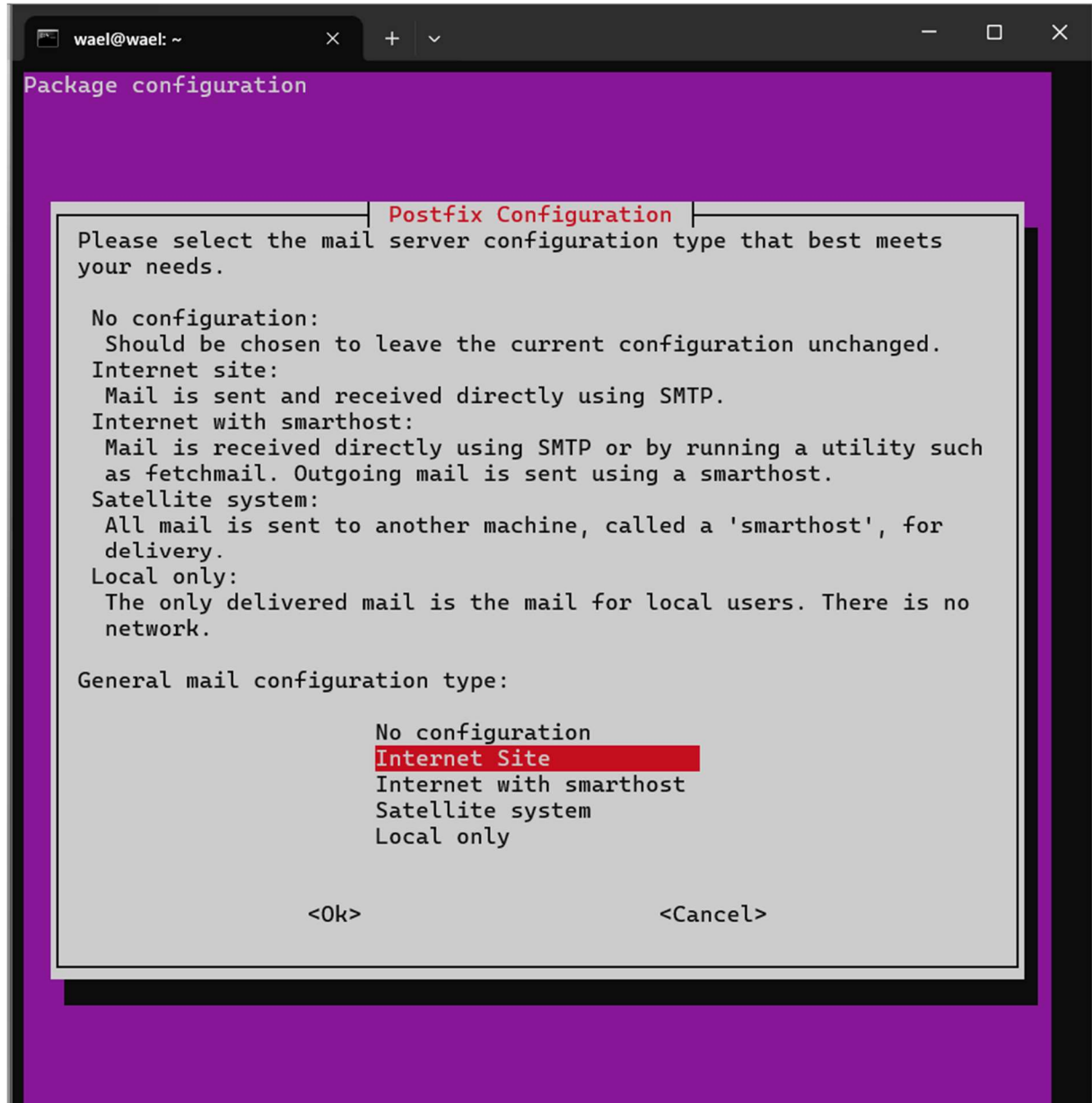
Install Postfix

sudo apt update

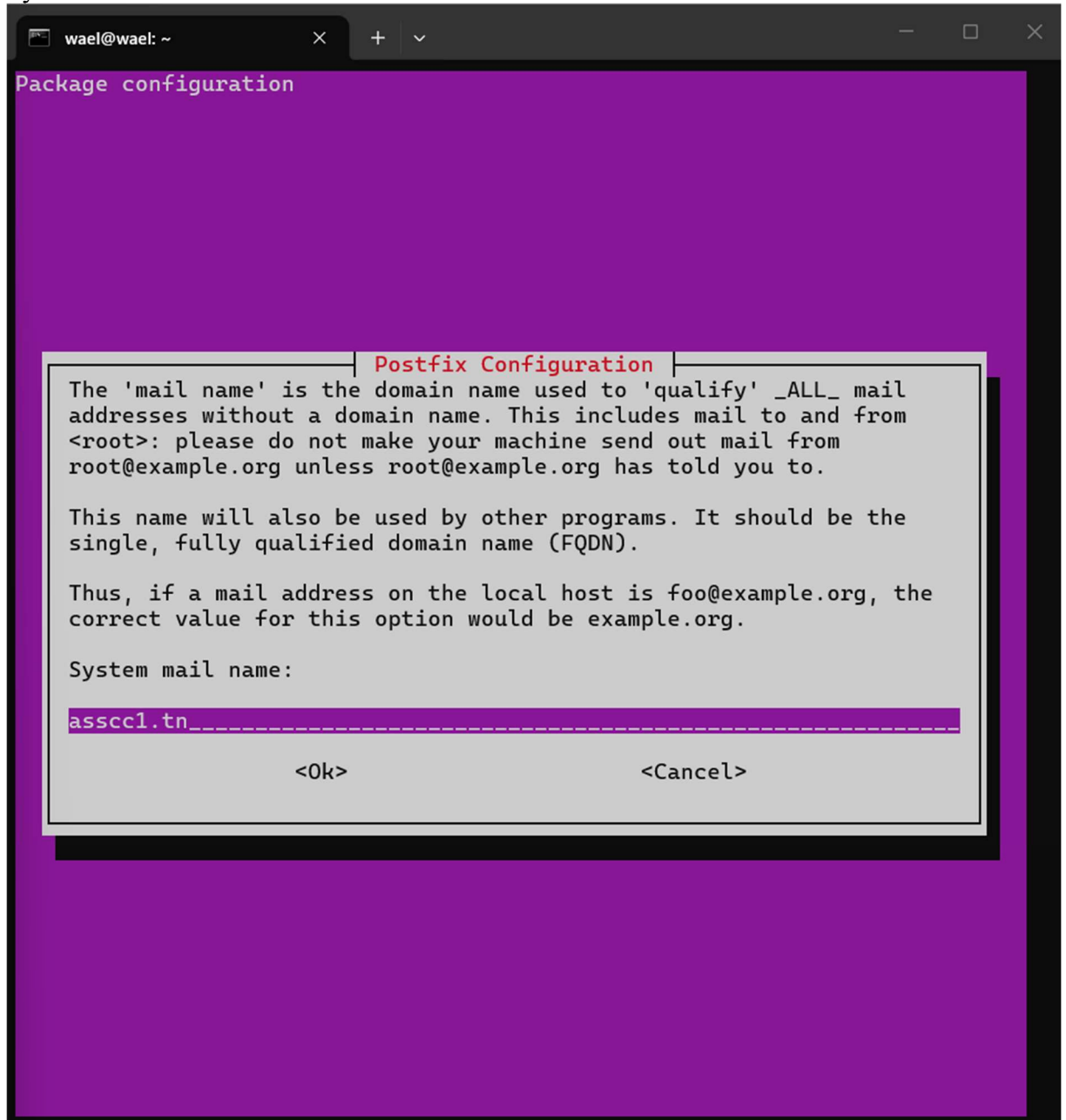
sudo apt install postfix mailutils -y

During installation:

- Select: "Internet Site"



- System mail name: asscc1.tn



Edit the main config: `sudo nano /etc/postfix/main.cf`

```
wael@wael: ~
GNU nano 7.2 /etc/postfix/main.cf

smtp_tls_CApath=/etc/ssl/certs
# change this
smtp_tls_security_level=encrypt
smtp_tls_session_cache_database = btree:${data_directory}/smtp_scache

smtpd_relay_restrictions = permit_mynetworks permit_sasl_authenticated defer_unauth_desti
myhostname = wael.home
alias_maps = hash:/etc/aliases
alias_database = hash:/etc/aliases
myorigin = /etc/mailname
mydestination = $myhostname, asscc1.tn, wael, localhost.localdomain, localhost
relayhost = [smtp-relay.brevo.com]:587
smtp_sasl_auth_enable = yes
smtp_sasl_password_maps = hash:/etc/postfix/sasl_passwd
smtp_sasl_security_options = noanonymous
mynetworks = 127.0.0.0/8, 192.168.1.0/24
smtpd_recipient_restrictions = permit_mynetworks, reject_unauth_destination
mailbox_size_limit = 0
recipient_delimiter = +
inet_interfaces = all
inet_protocols = all
myorigin = asscc1.tn
smtp_generic_maps = hash:/etc/postfix/generic

^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute    ^C Location
^X Exit      ^R Read File  ^_ Replace    ^U Paste      ^J Justify    ^/_ Go To Line
```

```
relayhost = [smtp-relay.brevo.com]:587
smtp_sasl_auth_enable = yes
smtp_sasl_password_maps = hash:/etc/postfix/sasl_passwd
smtp_sasl_security_options = noanonymous
smtp_tls_security_level = encrypt
mynetworks = 127.0.0.0/8, 192.168.1.0/24
smtpd_recipient_restrictions = permit_mynetworks, reject_unauth_destination
myorigin = asscc1.tn
smtp_generic_maps = hash:/etc/postfix/generic
```

Add my credentials: `sudo nano /etc/postfix/sasl_passwd`

`sudo nano /etc/postfix/sasl_passwd`

Secure and Activate:

`sudo chmod 600 /etc/postfix/sasl_passwd`

`sudo postmap /etc/postfix/sasl_passwd`

`sudo systemctl restart postfix`

`sudo nano /etc/postfix/generic`

wael@wael blhwdywayl@gmail.com

Test it:

```
wael@wael: ~
wael@wael:~$ echo "Test from OwnCloud server" | mail -s "Working Now" waelbalhoudi@gmail.com
wael@wael:~$
```

3. Mise en place de GrayLog

step1 : create a docker-compose.yml

wael@wael:~/graylog\$ cat docker-compose.yml

version: "3.8"

services:

mongodb:

image: mongo:6.0.18

container_name: graylog_mongodb

restart: always

volumes:

- mongodb_data:/data/db

- mongodb_config:/data/configdb

networks:

- graylog

opensearch:

image: opensearchproject/opensearch:2.15.0

container_name: graylog_opensearch

restart: always

environment:

- OPENSEARCH_INITIAL_ADMIN_PASSWORD=BlueSky#2026Server!

- discovery.type=single-node

- plugins.security.disabled=true

- OPENSEARCH_JAVA_OPTS=-Xms1g -Xmx1g

- bootstrap.memory_lock=true

- action.auto_create_index=false

ulimits:

memlock:

soft: -1

hard: -1

nofile:

soft: 65536

hard: 65536

volumes:

- opensearch_data:/usr/share/opensearch/data

networks:

- graylog

graylog:

image: graylog/graylog:6.1

container_name: graylog_server

restart: always

depends_on:

- mongodb

- opensearch

entrypoint: /usr/bin/tini -- wait-for-it opensearch:9200 -- /docker-entrypoint.sh

environment:

- GRAYLOG_TRANSPORT_EMAIL_ENABLED=true

- GRAYLOG_TRANSPORT_EMAIL_HOSTNAME=host.docker.internal
- GRAYLOG_TRANSPORT_EMAIL_PORT=25
- GRAYLOG_TRANSPORT_EMAIL_USE_AUTH=false
- GRAYLOG_TRANSPORT_EMAIL_USE_TLS=false
- GRAYLOG_TRANSPORT_EMAIL_FROM_EMAIL=blhwdywayl@gmail.com
- GRAYLOG_TRANSPORT_EMAIL_SUBJECT_PREFIX=[GRAYLOG-TP]
- GRAYLOG_PASSWORD_SECRET=\${GRAYLOG_PASSWORD_SECRET}
- GRAYLOG_ROOT_PASSWORD_SHA2=\${GRAYLOG_ROOT_PASSWORD_SHA2}
- GRAYLOG_HTTP_BIND_ADDRESS=0.0.0.0:9000
- GRAYLOG_HTTP_EXTERNAL_URI=\${GRAYLOG_HTTP_EXTERNAL_URI}
- GRAYLOG_ELASTICSEARCH_HOSTS=http://opensearch:9200
- GRAYLOG_MONGODB_URI=mongodb://mongodb:27017/graylog

ports:

- "9000:9000"
- "1514:1514/tcp"
- "1514:1514/udp"
- "12201:12201/tcp"
- "12201:12201/udp"

volumes:

- graylog_data:/usr/share/graylog/data

extra_hosts:

- "host.docker.internal:host-gateway"

networks:

- graylog

networks:

graylog:

driver: bridge

volumes:

mongodb_data:

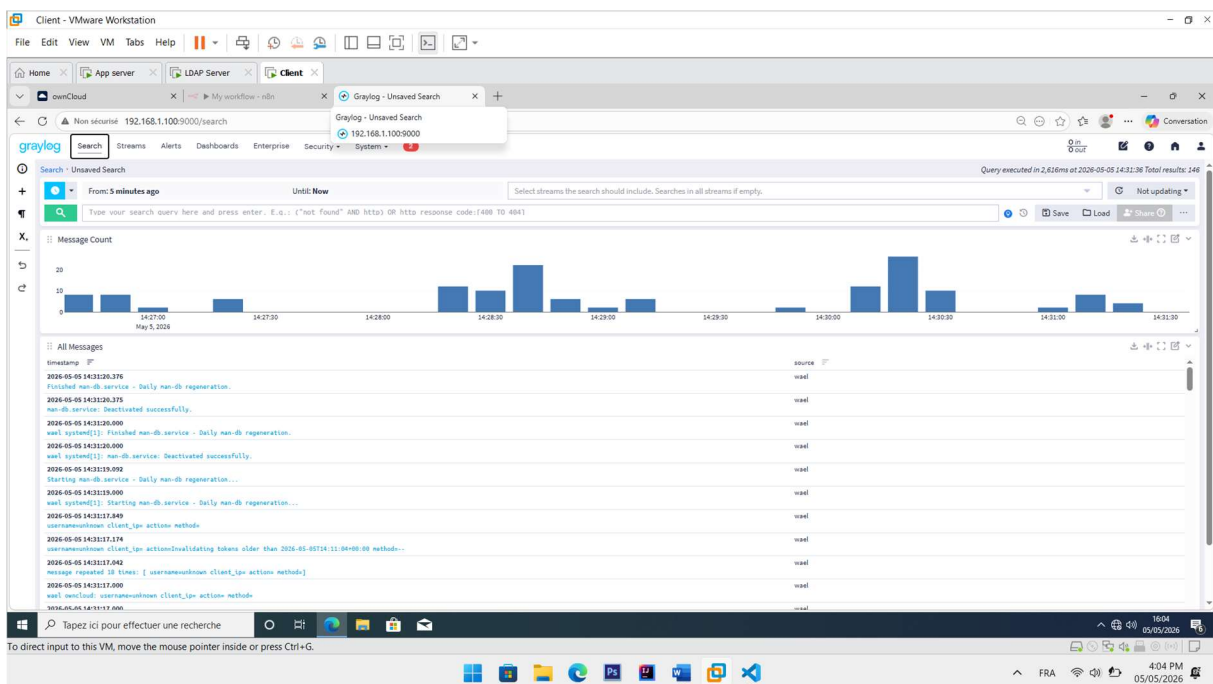
mongodb_config:

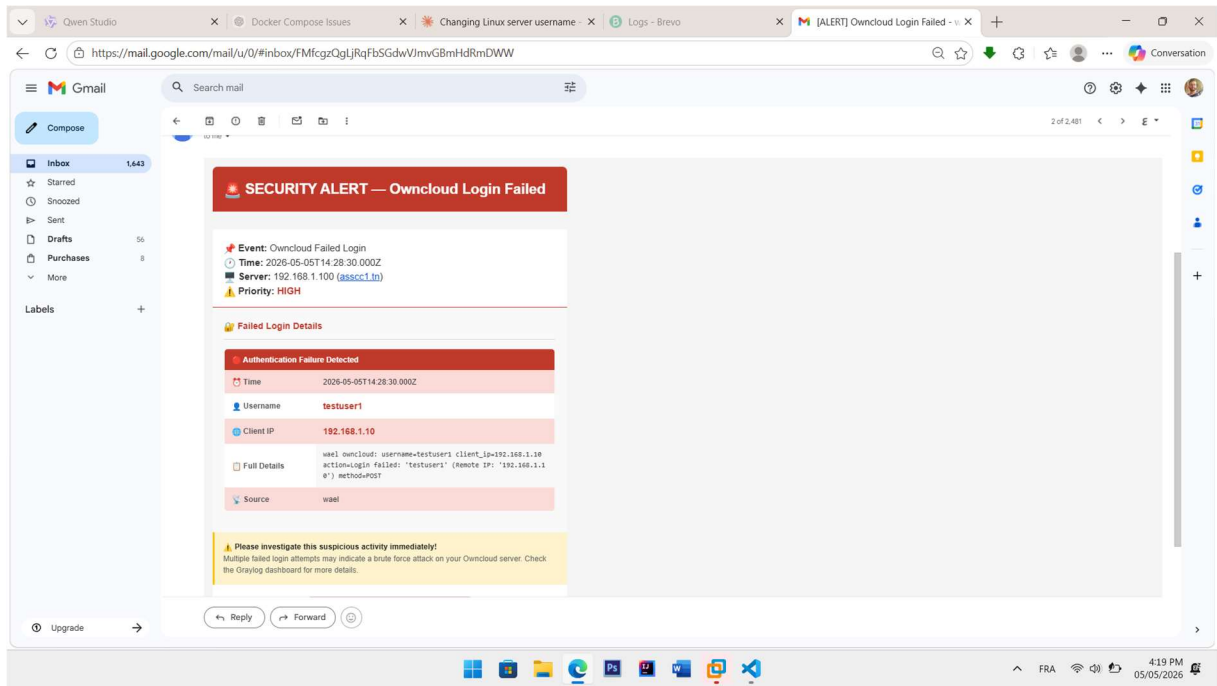
opensearch_data:

graylog_data:

and run it using `compose up -d`

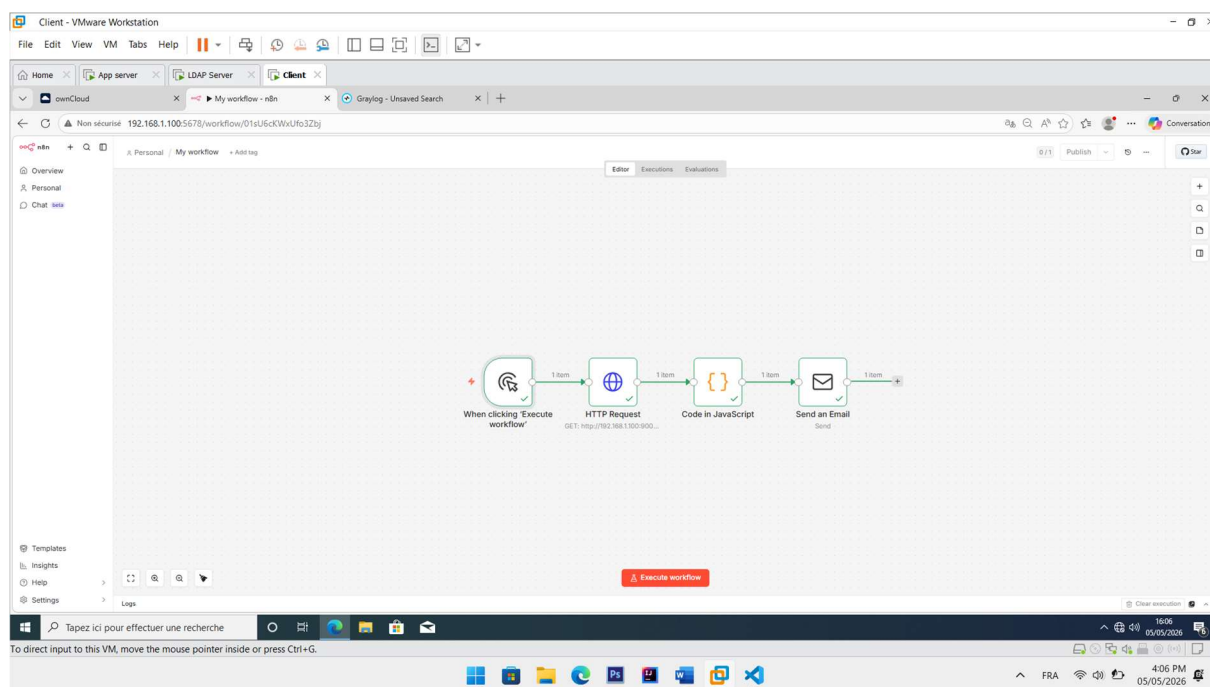
```
wael@wael:~/graylog$ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
ee393618ce5f   n8nio/n8n                           "tini -- /docker-ent..." 9 hours ago    Up 57 minutes    0.0.0.0:5678->5678/tcp, [::]:5678->5678/tcp
712de3e39c49   graylog/graylog:6.1                "/usr/bin/tini -- wa..." 39 hours ago   Up 57 minutes (healthy)    0.0.0.0:9000->9000/tcp, 0.0.0.0:1514->1514/udp, [::]:9000->9000/tcp, [::]:1514->1514/udp, 0.0.0.0:12201->12201/tcp, 0.0.0.0:12201->12201/udp, [::]:12201->12201/tcp, [::]:12201->12201/udp
1f2211fc05b0   mongo:6.0.18                       "docker-entrypoint.s..." 39 hours ago   Up 57 minutes    27017/tcp
61bf28dfdf6    opensearchproject/opensearch:2.15.0 "/.opensearch-docker..." 39 hours ago   Up 57 minutes    9200/tcp, 9300/tcp, 9600/tcp, 9650/tcp
df7959e809f4    owncloud/server:10.13              "/usr/bin/entrypoint..." 3 days ago     Up 57 minutes    0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp
wael@wael:~/graylog$
```





5. Automatisation avec N8N (Extension)

- Déploiement via Docker
- Création de workflows
- Intégration avec Graylog (webhook)
- Génération de rapports automatiques
- Envoi d'emails via Postfix



The screenshot shows a Gmail inbox with an email titled "Daily Failed Login Report" from `bhwidyway@1140407.brevo.send.com`. The email content includes a table of failed login attempts.

Daily Failed Login Report
Date: 2026-05-05
Total Failed Logins: 14

Time	Username	IP Address	Action
2026-05-05T14:28:30.818Z	testuser1	192.168.1.10	Login failed: 'testuser1' (Remote IP: '192.168.1.10') method=POST
2026-05-05T14:28:30.000Z	testuser1	192.168.1.10	Login failed: 'testuser1' (Remote IP: '192.168.1.10') method=POST
2026-05-05T04:25:30.938Z	waef	192.168.1.10	Login failed: 'waef' (Remote IP: '192.168.1.10') method=POST
2026-05-05T04:25:30.000Z	waef	192.168.1.10	Login failed: 'waef' (Remote IP: '192.168.1.10') method=POST
2026-05-05T04:25:22.291Z	admin	192.168.1.10	Login failed: 'admin' (Remote IP: '192.168.1.10') method=POST
2026-05-05T04:25:22.000Z	admin	192.168.1.10	Login failed: 'admin' (Remote IP: '192.168.1.10') method=POST
2026-05-05T04:25:15.282Z	admin	192.168.1.10	Login failed: 'admin' (Remote IP: '192.168.1.10') method=POST
2026-05-05T04:25:15.000Z	admin	192.168.1.10	Login failed: 'admin' (Remote IP: '192.168.1.10') method=POST
2026-05-05T04:25:13.532Z	admin	192.168.1.10	Login failed: 'admin' (Remote IP: '192.168.1.10') method=POST
2026-05-05T04:25:13.000Z	admin	192.168.1.10	Login failed: 'admin' (Remote IP: '192.168.1.10') method=POST
2026-05-05T04:22:20.097Z	testuser1	192.168.1.10	Login failed: 'testuser1' (Remote IP: '192.168.1.10') method=POST
2026-05-05T04:22:20.000Z	testuser1	192.168.1.10	Login failed: 'testuser1' (Remote IP: '192.168.1.10') method=POST
2026-05-04T15:26:01.357Z	testuser1	192.168.1.10	Login failed: 'testuser1' (Remote IP: '192.168.1.10') method=POST
2026-05-04T15:26:01.000Z	testuser1	192.168.1.10	Login failed: 'testuser1' (Remote IP: '192.168.1.10') method=POST